

e them to support their team members in setting personal development goals and creating shared expectations, and ensure consistency in our expectations for different levels across the organization.

Having Job frameworks that managers and team members share will help us make things more transparent, consistent, actionable, and equitable. Departments leads can work with their People Business Partner to build out specific functional frameworks with job-specific competencies beyond the general frameworks provided here.

How to use Job Frameworks

We are using the Job Frameworks in the following programs:

- Promotion Process
 - Here the review of the Job Framework will be required ahead of the quarterly Department Promotion Calibrations.
 - The Job Framework will help provide focus for the promotion document to ensure core areas of performance at the next level are captured consistently across the company
- Career Development Conversations
 - Both managers and team members can leverage the frameworks in their conversations.
 - The frameworks drive transparency of the competencies and job criteria of different levels at GitLab. When aligned with the Career Development goals of the team member, the team member and manager can collaborate on developing these competencies by leveraging Learning & Development Programs
- Organizational Design and Headcount Planning
 - The frameworks will support Leadership and hiring teams in identifying the appropriate levels for to be opened roles.
 - For expanding and introducing new job levels we will leverage the frameworks to review what distinctive competencies are that we would need.
- Talent Assessment:
 - Job frameworks should be leveraged in the Talent Assessment program for both the self-assessment and the manager assessment in evaluating a team member against the competencies for their grade level. In a review of the competencies per level, strengths and development opportunities may surface which can help with content for the review that can lead to discussions with the team member and their manager on future development and career opportunities.
- Succession Planning:
 - We leverage Job Frameworks in Succession planning as we assess requirements for each role level.

Competencies per Job

For each Job Framework we have identified categories for describing competencies we expect to see at each level. Below we describe the definitions of each category:

- Scope & Influence: The scope of the responsibilities and ways each job influences product, team members or company strategy.
This ranges from focus on own work to cross-company and external influence in terms of product, team members, customers or company strategy.
- Complexity & Problem solving: The level of complexity and problem-solving skills in day-to-

day responsibilities and projects.

This ranges from low/moderate complexity and problem solving to highly complex issues that influence the accomplishment of long-term goals of GitLab.

- Leadership & People Management/Communication: The level of leadership they display and how they communicate within the organization. We expect that individual contributors also show leadership in their roles.

This ranges from within their team to executives and board members.

- High Performing Teams: Each characteristic of high performance is broken into 3 indicators to help you recognize and measure success.

- Values Competencies: Competencies that are aligned with our CREDIT Values.

- Remote Working Competencies: Competencies that are aligned with our Remote working competencies.

- Functional Competencies: Competencies that are specific per function. These are built out by each function themselves.

Besides the above competency categories we have added two categories to the Job Frameworks:

- Typical Reporting Structure: This shows the typical reporting structure for that Job level.

- Distinction with next level: This is a brief statement of what distinguishes one level from the next level.

Individual Contributor Job Framework

Individual contributors are Managers of One. ICs make up the majority of the GitLab team members. The Job Framework for ICs is a high level guide that describes the minimum competencies we expect at each level of an IC. For exact requirements and responsibilities per level team members should refer to the Job Family page of each job.

People Managers Job Framework

You can find the Job Framework for People Managers in the tab here. Below we will describe each level and their reporting structure.

General Expectations of all People Managers

For all People Managers we have general expectations in behavior which we describe below. These are not captured in the Job Framework but rather can be used as a guide on how we expect people managers to behave and interact with their team/peers. The difference with the Job Framework is that it doesn't describe high level competencies but rather sets very concrete expectations and GitLab basics.

Managers

Managers belong to the Management Group. You can view the Job Framework for expectations per competency category. Typically a Manager reports to a Senior Manager or Director.

Senior Managers

Senior Managers belong to the Management Group. Furthermore you can view the Job Framework for expectations per competency category. Typically a Senior Manager reports to a Director or

above.

Director

Directors are typically managers of managers. They make up the Director Group. In some cost center, Directors map to departments. For example, under the G&A Division, Business Operations is a department led by a Director. This is not a hard-and-fast rule, though, as under G&A, People is all under one department.

Members of the director group are expected to demonstrate leadership in the way all members of the management group are, plus the additional competencies you can find in the Job Framework. Typically a Director reports to a Senior Director or above.

Director group

Members of the director group are expected to demonstrate leadership in the way all members of the management group are, plus:

1. They have the ability to align the day-to-day execution to the top objectives of the company, and they are responsible for making sure the top objectives are well communicated to team members.
1. They work peer-to-peer and sponsor healthy conflict amongst the team to resolve issues quickly, escalating only when all options are exhausted.
1. They ambitiously define roles for, grow and hire their teams for what is needed from the business in the next 3-4 years.
1. They coach their teams to work within the communication guidelines and lead by example.

Senior Directors

Senior leaders are defined as Senior Directors or VPs. They are all members of our S-group. Senior Directors are typically managers of Managers and/or Directors. In some divisions, senior leaders map to departments in the cost center structure.

Members of the S-group are expected to demonstrate leadership in the way all members of the director group are, plus the additional competencies you can find in the Job Framework. Typically a Senior Director reports to a VP or above.

S-group

Members of the S-group are expected to demonstrate leadership in the way all members of the director group are, plus:

1. They have significant strategic and functional responsibility.
1. They have significant operating budget responsibility (things like compensation planning, budget allocation, negotiation, investment tradeoff decisions).
1. They have leverage and influence over the performance and success of large teams (both reporting directly and indirectly) and their success will result in increased success across large numbers of people.
1. The impact of their decisions are broad and significant.
1. Some of them have critical external responsibility to represent the company and make decisions and statements for which the company is accountable.

VP

VP's are members of our S-group. VP's are typically managers of Senior Directors and/or Directors. Similar to Senior Directors they map to departments.

Typically a VP reports to an Executive.

VP-Directs

VP-Directs are a subset of GitLab VPs who report directly to a member of E-Group. They are members of the VP-Directs Group.

Executives

The executive layer is structured as follows. R&D focused executives include the Chief Product Officer (CPO), Chief Information Security Officer (CISO) and the Chief Technology Officer (CTO).

Go-to-market focused executives include the Chief Revenue Officer (CRO) and Chief Marketing Officer (CMO).

The three enabling functions, legal, finance and people, also each have a C-level executive, the Chief Legal Officer (CLO), Chief Financial Officer (CFO) and Chief People Officer (CPO).

Together, these executives consist of the E-group

They meet weekly, attend quarterly board meetings, have a public Slack channel e-group for most discussion topics, as well as a private one for rare confidential matters.

Except for Sales and Marketing, there are usually multiple executives to a cost center. For example, CLO, CFO, CEO, and CPO all fall under the G&A (General & Administrative Expenses) Cost Center.

E-group

Members of the E-group are expected to demonstrate leadership in the way all members of the S-group are, plus:

1. They suggest relevant, ambitious, and quantifiable OKRs and achieve 70% of them.
1. They are reliable and ensure their teams complete what they agreed to do.
1. They are proactive about detecting and communicating problems in their functions before other departments even notice them.
1. They hire and retain leaders that perform better in their functional areas.
1. They create roles and set requirements for what is needed 3-4 years out and hire for that profile.
1. They get a huge amount of things done by iterating quickly and training their department in iteration.
1. They define and communicate the business strategy and vision, instead of being overly tactical in the business.
1. They share insights about other functional areas that make others better at their job.
1. They suggest and implement improvements to our cross-functional processes.
1. They frequently help achieve results outside their function.
1. They make other executives better in their discipline

Management group

Members of the management group are expected to demonstrate leadership in the way all GitLab team members are, plus:

1. Ensuring team members feel included and valued is one of the most important tasks of a manager. Proactively create psychological safety with your team members so that diverse perspectives can be heard and everyone can communicate and contribute authentically and creatively.
1. Ensuring team members understand what is expected of them in their roles is a critical role that managers have to ensure company success. Managers should ensure job families include specific performance indicators and that is clearly communicated to each team member.
1. Managing underperformance is another important task of a manager.
1. When times are great, be a voice of moderation. When times are bad, be a voice of hope. In order to effectively be the voice of hope, managers should make sure they understand the company mission, goals, and leadership decisions. If managers have concerns about leadership decisions, they should voice them to leaders in order to understand the context, share key insights from their team members to ensure we make sound decisions, and then be able to explain context to their team members.
1. To maintain an effective organization, a manager's span of control should be around 7 people, ranging anywhere from 4 to 10. Below this range the inefficiency of an extra organizational layer is larger than the benefit of a specialized group. Above this range, the manager doesn't have time to do proper 1:1s anymore.
1. The span of control is the same at every level of the organization. This prevents the organization from having an additional layer that adds cost and reduces decision making speed. To increase the number of reports you can handle increase delegation to the directly responsible individual who can be a manager. Size the number of reports on where you expect the organization to be in 6 months from now.
1. When you praise someone, try to do it publicly and in front of an audience. When you give suggestions to improve, do it privately 1 on 1. A Harvard study has shown that the ideal ratio of positive to negative feedback for high-performing teams is nearly 6:1. Be generous with your positive feedback.
1. Express gratitude in team meetings, group conversations, and other communications to people who constructively challenge your opinions and ask difficult questions. This helps create psychological safety, promote our values, and prevent the five dysfunctions.
1. Understand that there are different ways to get to the same goal. There are different perspectives, and discussions need to happen.
1. When someone says they are considering quitting, drop everything and listen to them. Ask questions to find out what their concerns are. If you delay, the person will not feel valued and the decision will be irreversible.
1. In addition to announcing new team member arrivals, departures are also announced in the team-member-updates chat channel (but only after the Google/Slack accounts are revoked, see the offboarding page and the offboarding checklist for details). We must respect the privacy of the individual concerned. If you are asked why someone has left or is leaving, please refer that person to the general guidelines section of the handbook where we describe what can and cannot be shared.
1. People should not be given a raise or a title because they ask for it or threaten to quit. We should pro-actively give raises and promote people without people asking. If you do it when people ask, you are being unfair to people that don't ask and you'll end up with many more people asking.

1. Don't refer to GitLab as a family. It is great that our team feels like a close-knit group and we should encourage that, as this builds a stronger team. But families and teams are different. Families come together for the relationship and do what is critical to retain it. Teams are assembled for the task and do what is required to complete it. Don't put the relationship above the task. Besides, families don't have an offboarding process. Families should have unconditional love, while teams have conditional love. The best companies are supporters of families.. The idea that your team is your family can lead to unwarranted pressure: "this office is like a family... in that there is vague pressure to be here on holidays". You are the CEO of your own career, treat GitLab as a customer.
1. Praise and credit the work of your reports to the rest of the company, never present it as your own. This and many other great lessons can be found in an Ask MetaFilter thread worth reading.
1. Try to be aware of your cognitive biases.
1. Combine consistency and agility.
1. Consider maintaining a README file for your teams which documents your expectations and working style. Encourage MRs to it.
1. Although our time scales are shorter, there is a great article about how to think about PIPs.
1. Do everything to unblock people. If someone has a question that is keeping them from being productive, try to answer the question yourself or find someone who can.
1. Communicate in a professional manner as though your words will be shared widely (e.g. published in a newspaper).
1. Employ multimodal communication to broadcast important decisions.
1. You are expected to respond on social media. If you have questions, please seek counsel in the socialmedia chat channel.
1. Make sure your reports experience a sense of progress in their work.
1. A tweet by Sam Altman combined with a reply by Paul Graham says it best: "People either get shit done or they don't. And it's easy to be tricked because they can be smart but never actually do anything." Watch for results instead of articulate answers to questions, otherwise you'll take too much time identifying under-performers.
1. GitLab Contribute (formerly GitLab summits) is meant for informal communication and bonding across the company. There is a limited time for business activities during GitLab Contribute so all our regular meetings should happen outside of it. We want informal, cross team, and open-ended meetings that include individual contributors. For example, inviting everyone to suggest currently missing functionality in GitLab.
1. Never delay a decision until GitLab Contribute. Instead, use them as a deadline to get things done earlier.
1. We don't have explicit 20% time at GitLab. We measure impact and not activity. If people are getting good results in the work that is assigned to them they are free to contribute to other parts of the company or work on a pet project. Don't say, "Your work on that pet project is hurting your performance." Instead, say, "We agreed to get X done but it is delayed, what happened and how can I help?"
1. Pick a metric before launching something new. 9 out of 10 launches fail. If a project is not working out shut it down completely. Starving a team of headcount to have it die a slow death is not frugal nor motivating. Fund the winners which will still take years to break even.
1. Do not discuss raises in advance because the salary calculator may change before the amount of the raise is decided.
1. Do not discuss promotions, salary increases, or bonuses until the changes have gone through the entire approval process. People Ops will inform the manager when they can inform the team member of the promotion and increase.

1. Instead of prescribing a direction to your reports, it is best to ask questions following the Socratic method until you're happy with the direction. Your reports will have deeper knowledge in a more narrow area, so it is easy to draw different conclusions because they base theirs on different data. That is why the questions are so important.
1. Follow Berkshire's common injunction: "Always tell us the bad news promptly. It is only the good news that can wait." Make sure to inform your manager of bad news as quickly as possible. Promptly reporting bad news is essential to preserving the trust that is needed to recover from it.
1. Try to avoid military analogies and imagery. We're not an army, we're not at war, there is no battle, we're not killing anyone, and we don't have weapons. Military language is not inclusive and can lead to zero sum thinking. We take competing and winning very seriously, but there is no need to describe things using vocabulary of physical violence. Similarly, non-collaborative and aggressive terms like "rock star" and "badass" put up walls between people. If a term is standard in the industry, for example killing a Unix process, it is acceptable to use it because that is more efficient. Do use "primary-secondary" instead of "master-slave" for replication mechanisms.
1. Complain up and explain down. Raise concerns you hear to your manager. When peers or reports complain, explain why a decision was made. If you don't understand why, ask your manager.
1. Create empathy for decisions from other leaders that negatively impact your team by explaining the reasons behind them. Organize a recorded AMA session with your team and the other leaders and encourage your team (as well as yourself) to ask any unanswered questions. Lead by example by ensuring that the discussion exposes what is best for the organization as a whole. Never present yourself as protecting the team from the rest of the organization; this creates a siege mentality and hinders collaboration.
1. Coach team members to establish good remote work practices, by encouraging asynchronous communication, being handbook-first, communicating poor audio or video quality, or pointing out an open microphone.
1. Ensuring that GitLab's policies are upheld is an essential part of a manager's responsibility. Managers have an obligation to look out for both their team members and the business's best interest. If a manager becomes aware that a team member is in violation of a GitLab policy (including but not limited to the GitLab Code of Business Conduct and Ethics and Relocation policy as examples), it is their responsibility to communicate this to the aligned People Business Partner or Legal team immediately.

Board Members

Board Members serve on the GitLab board and participate in board meetings and board committees, as well as other responsibilities.

CEO Skips

You may occasionally hear of the E-Group hosting an "CEO Skips" call or meeting. The CEO Skips group is made up of anyone who reports directly to the e-group. It also includes People Business Partners, Chief of Staffs, and internal communications. It is made up of some ICs, some managers, some directors, and some senior leaders.

Functional Leaders

Functional Leaders include all CEO Skips and a small number of other team members who are invited by E-Group members.

This group is called on to help provide input and communicate messaging when appropriate.

As an example, the Functional Leaders meets after every e-group offsite.

Directs-Group

Directs-Group is a group made up of a senior leader from each function. These leaders operate as extensions of E-Group and support key initiatives within a six to nine month rotation. The Office of the CEO is responsible for organizing and orienting this group in collaboration with the cohorts E-Group Sponsor.

Directs-Group Program Objectives

1. Give greater exposure to E-Group and E-Group decision making to develop leaders with C-level potential
1. Enlist a larger group of people to help drive and inform key initiatives
1. Support cascade of information from E-Group within their teams and functions. This would include cascading to peers on their functional leadership team.

Directs-Group Responsibilities

1. Attend portions of and contribute in designated meetings with E-Group and E-Group Offsites
1. Participate in onboarding, offboarding, regularly scheduled Directs-Group syncs, professional development programming, and ramping of the next cohort
1. Support communication cascading and initiative support resulting from your engagement in Directs-Group
1. Serve in this role while managing existing responsibilities
1. Contribute to the cohort project

Directs-Group Selection Process

1. The Office of the CEO kicks off the selection process three months in advance of every cohort end date to allow adequate time for Directs-Group identification and onboarding in advance of first E-Group Offsite and appropriate transition from one Directs group to another.
1. Each E-Group member identifies first and second choice candidates. E-Group will discuss if TMRG criteria is not met within first choice candidate selections.
1. Candidates will be identified and notified at least 45 days before the start of the new cohort in which they would serve in the role.
1. If a candidate declines their nomination, the function leader can nominate a new candidate for consideration.

Directs-Group Selection Criteria

Each Directs-Group cohort must meet the following criteria:

1. High growth and high performing team members in good standing
1. Director + who are direct reports to an E-Group member
1. Folks who consistently demonstrate CREDIT Values
1. · or more of each cohort needs to be comprised of members from underrepresented groups
1. Include someone who is based outside of the United States

Directs-Group leadership

During the Directs-Group nomination process, E-Group will nominate a person to lead the cohort. Ideally this person comes from the previous cohort so they have experience. It is this person's responsibility to:

1. Get agreement between E-Group and Directs-Group on key deliverables
1. Ensure that the group functions well
1. Be an official member of the incoming cohort
1. Help ramp up the incoming cohort
1. Coordinate incoming team activities and deliverables
1. Serve as a cohort representative with E-Group
1. Ensure that the group, in collaboration with the E-Group sponsor, achieves success in at least one cohort project

This person will be selected based on:

1. Their demonstrated ability to lead a cross-functional leadership team
1. Ability to make this additional time commitment

Directs-Group Timing

The cohorts span six to nine months aligning with two to three quarters in GitLab's fiscal calendar. Specific start dates are timed to align with E-Group Offsites.

E-Group Sponsor for Directs-Group

Each cohort will have an E-Group sponsor for the duration of the cohort. This person will attend Directs-Group Meetings and serve as a formal link to the broader E-Group. This person will be a stable counterpart to the Directs-Group Leader. The E-Group sponsor and Directs-Group Leader will come from different functions. The Sponsor will also assist Directs-Group in identifying at least one area in which they can contribute as a cohort. During Directs-Group meetings, the Sponsor will check on initiative status and offer feedback.

Directs-Group Participants

Past, present and future Directs-Group Participants will be listed here.

| Term | Directs-Group Cohort Members |

|-----|-----|

| 2022-01 to 2022-10 | Urja Patel, Matt Taylor, Rob Allen, Melissa Smolenksy, Christopher Lefelhocz, Kenny Johnston, Justin Farris |

| 2022-11 to 2023-08 | Jim Gladen, Pattie Egan, Dave Steer, Hillary Benson, Eliran Mesika (cohort lead), Stan Hu, Christie Lenneville, Ryan O'Neill, Joaquin Fuentes, Sid Sijbrandij (E-Group Sponsor) |

Cohort project

Each Directs-Group will have at least one cohort project. Cohort projects should be cross-functional initiatives designed to improve GitLab in some way. At least one cohort project

should be identified within the first month of the cohort. Selection should involve Directs-Group and the E-Group Sponsor. Cohorts can recommend project ideas for incoming cohorts.

Cohort Projects:

1. 2022-01 to 2022-10 : Stop Initiative, Internal Handbook updates
1. 2022-11 to 2023-08 : Strategic plan refresh

VP-Directs Group

VP-Directs Group is largely made up of VP-Directs, VPs who report directly to members of E-Group. This is a senior group of leaders who are able to assist in the cascading of critical communications, support change management, ensure our mission & vision continues to be thoroughly understood at all levels of the organization, support in top priority cross-functional activities, and act as a gauge of company sentiment & engagement. In cases in which a function has 0-2 VP-Directs, an E-Group member may nominate 1 or 2 additional people from within the function to help represent the function.

VP-Directs Group Objectives

Through monthly meetings and a Slack channel, this group will be engaged to:

1. Cascade information: E-Group has a specific message that the organization needs to absorb and understand and senior leaders can help in communicating the message. This is particularly important when heavy change management is required for a specific issue or challenge and E-Group needs broader leadership support. We'd ask this group to absorb it and drive the change through their organizations.
1. Provide information and share perspectives: This group can be a senior sounding board as decisions are made or actions are being taken. For example, this group could be engaged when E-Group has identified a need for a strategic organizational pivot and broader senior level input is desired.
1. Provide cross-functional initiative leadership: E-Group may ask this group to take on specific cross-functional initiatives. For example, this group could play a role in addressing feedback from the Employee Engagement Survey (alongside all functional leaders) or support strategic planning activities.
1. Hear from customers: We'll invite customers to speak to this group, so we can, as a leadership group, understand their perspective on GitLab and our challenges and opportunities and continue to drive customer-centricity throughout the organization.

VP-Directs Group Commitment

1. Participate in monthly meeting, synchronously when possible (recorded meeting, if appropriate)
1. Monitor the Slack channel
1. Share input on proposals and plan
1. Reach out to program managers with any ideas on what should be discussed or how to improve group effectiveness

VP-Directs Group Management

Initial management will include:

1. Rotating E-Group sponsor: formal E-Group representative
1. The Chief of Staff to the CEO: owns E-Group interlock, partners with sponsor
1. VP of Talent and Engagement: Owns team engagement and partners with VP of Global communications on internal communications. VP of Global communications owns broader communication where necessary.

This initial management team will be responsible for getting the group up and running and starting to establish rigor & routine. We can iterate from this model as we better understand this

The management team will:

1. Coordinate with the EBA team to ensure that invite series are sent and established
1. Ensure appropriate actions are noted and followed up on, hold accountability where necessary
1. Serve as an intersection between E-Group and VP Group to ensure that meetings have a clear purpose, information is being effectively shared, and group goals are being met

Organizational Structure

- Divisions: the area under one executive. e.g. the Engineering division.
- Departments: lead by Directors or VPs and comprise multiple teams or sub-departments e.g. the Infrastructure department within the Engineering division
- Sub-departments: lead by Directors or Senior Managers and comprised of multiple teams e.g. the Enablement Sub-department within the Development department
- Compartments: lead by Senior Managers and comprised of multiple teams where the Sub-department structure is already used e.g. the Manage Compartment with the Dev Sub-department
- Teams: constitute departments and are made of a line manager and their direct reports e.g. the Security operations team within the Security Department

Note - within the Engineering and Product divisions we try to maintain a close relationship between our organizational structure and our Product Hierarchy in order to maintain stable counterparts in our organizational structure.

Finance also has a notion called "departments" for financial planning purposes. But these do not align with our organizational departments. For instance the finance department "product development" rolls up both the PM and Engineering functions. But it excludes the Support department, which is part of the engineering function, but a different budget. This name collision should probably be resolved in the future. For further reference see our department roll up structure for accounting purposes.

We try our best to keep our handbook and documentation up to date, but certain terms used in the past have been updated through time. The term Functional Group is no longer used, and it currently is encompassed by the term Departments. The term Functional Group leader is no longer used, and it is currently encompassed by the term E-group leaders.

Organized by Output

In many ways, we are organized by output.

This way we can ensure that responsibilities don't overlap.

We also ensure every division has a clear priority.

Division	Output
Marketing	Generate Pipeline
Sales	Close Pipeline
Product	Prioritize development
Engineering	Execute development
People	Enable people
Finance	Ensure correctness
Legal	Ensure compliance
Security	Enable trust

Product Groups{product-groups}

Our engineering organization is directly aligned to groups as defined in product category hierarchy.

Our groups operate on the principle of stable counterparts from multiple functions.

For example, we have a Product Manager, Product Marketing Manager, Engineering Manager, Content Marketer, Backend Developers, Frontend Developers, and Product Designers that are all dedicated to a group called "Package". Collectively, these individuals form the "Package group". The word "Package" appears in their titles as a specialty, and in some cases, their team name.

We distinguish between types of stable counterparts to these Product Groups with:

- Primary Stable Counterparts - Team members assigned to our Product hierarchy (typically groups) from Product, Development, and Product Design.
- Complete Stable Counterparts - All team members assigned to product hierarchy from functions outside of the primary functions and defined in our product categories page. For example - we assign stable counterparts from Support, Product Marketing and Customer Success who are all considered part of the complete stable counterparts.

A group has no reporting lines because we don't want a matrix organization.

Instead, we rely on stable counterparts to make a group function well.

In some shared functions, like design, technical writing and quality individuals are paired to one or more stages so that there are stable counterparts.

While everyone can contribute, the scope of a group should be non-overlapping and not a required dependency for other groups to deliver value to users.

This facilitates results, iteration and efficiency.

Internal platform groups (those focused on a non-user facing part of our product, like a set of internal APIs) tend to create heavy coordination costs on other groups which depend on platform improvements to deliver valuable features to users. In order to stay efficient, it is important to ensure each group is non-blocking and is able to deliver value to users directly. This is why we avoid internal platform groups.

It is also important to ensure a group doesn't have a scope definition that is shared across multiple groups. Here are two examples:

1. We don't have an internationalization group.

That responsibility is shared across many groups.

We might instead have an internationalization tooling group.

1. We don't have a performance group.

Ensuring GitLab is performant is the responsibility of all groups.

We might instead have a performance tooling group.

We do have the Application Performance group and Database group whose charters involve consulting and providing frameworks for other teams, but are considered non-blocking.

The Application Performance group is focused on identifying systemic performance bottlenecks, creating documentation, and tooling to assist other groups in understanding and improving the performance of their features.

The Database group is focused on the specifics of database management/scaling and to provide consulting for development teams in need of database development guidance. While database related merge requests still require approval from a database maintainer our database review process has necessarily scaled beyond just the members of the database team.

Product Group health Assessment

The ability to execute the product roadmaps is dependent on the health of the product groups. Rather than rely on intuition, forceful personalities, or other ad-hoc methods, there are frameworks, such as the Drexler-Sibbet model, for developing high-performance teams. Furthermore, team health can be measured and improvements made systematically. For example, the Spotify Health Check model is a lightweight method of visualizing what can be improved within a team.

The goal of product group health assessment is to enable groups to identify improvement areas. If they are used to gauge the relative "maturity" of the group, it creates a perverse incentive for groups to stack the results to make them look as good as possible. As a result, it is important that health assessments should not be used by management to compare and contrast product groups against each other. Instead, the leaders of the product group are the DRIs to manage assessments and iterate on improvement over time.

To get started, feel free to make a copy of the Team Health Survey Template and edit to suit your development group's needs.

Building High Performing Product Groups Learning Session

The Learning & Development team can facilitate a live learning session with product groups to enable high performance. The Drexler-Sibbet model is a framework to lead teams through stages towards high performance. During the live learning session, a L&D facilitator will collaborate with a product group to define activities to align team members on what needs to occur to achieve high performance. If interested in scheduling a session, contact [learninganddevelopment](#) to learn more.

Setting Product Group for Team Members

Because it is our single source of truth (SSOT) for employee data, Workday serves as the SSOT for product group assignment. Each team member in R&D functions (Product/Engineering) is

assigned a specialty field in their team.yml entry. While this entry is editable in the www-gitlab-com project, any adjustments will be over-written by a daily sync from Workday. In order to adjust a team members specialty their manager must initiate a Job Information Change.

When designating a team members specialty we use the smallest unit of our Product hierarchy and their designated names. So for example:

- A team member who is a stable counterpart in the Code Review group in the Create stage would have the specialty designation of Create: Code Review
- A team member who is a stable counterpart to the entire Verify stage would have a specialty designation of Verify
- A team member who is a stable counterpart to the Access and Import groups in the Manage stage, and the Gitaly group in the Create stage would have specialty designations of Manage: Access, Manage: Import, and Create: Gitaly.

Single-Engineer Groups

The goal of a Single-Engineer Group (SEG) is to initiate GitLab into a planned or minimal category within the GitLab project. The single-engineer group is not to invest in an existing viable or complete category. Here is a list of product ideas that are candidates for a SEG to work on.

At GitLab, we believe in the power of a single engineer to accomplish amazing feats. Many open source projects started with a single engineer's decision to build around a problem they personally experienced. For instance, Continuous Integration by DZ and GitLab Runner by Kamil. We want to create room for this energy.

Our belief is that we can guarantee a higher rate of success by incubating ideas inside our larger organization and existing code base while limiting the negative aspects of friction that come from a larger organization. A few benefits of SEG include:

1. There are lots of decisions to be made, which happen more effectively in a single brain.
1. There is not enough code for multiple people to work on without running into merge conflicts.
1. Starting work earlier allows for more time for other people to contribute. We need to have a head start many years ahead of commercialization.

As a matter of process, we require that Single-engineer groups take part in our Software Demo Process as a way to collaborate asynchronously with stakeholders, get iterative feedback, and maintain at least minimal alignment with the rest of GitLab while keeping their autonomy

If the group finds great success, as measured by adoption, and needs to drive deeper category maturity then the next step is to consider forming a multi-person group. Sometimes the category is complete and the group can successfully dissolve. There are other times when the category does not yield the adoption. We will gather the lessons learned and consider either dissolving the group or investing further based on the learnings.

We have a huge opportunity in front of us and we want to be ambitious. Therefore over time we want to invest 10% of our R&D to continue to pursue these. However, in the spirit of iteration, we will start with only a handful of Single-engineer groups. As we learn more, and we get more

dollars to invest, we will gradually add more SEGs.

Success criteria

The success of the Engineer relates to getting a rapid, high-quality answer to the question of product-market fit. That answer can absolutely be "no", so performance is not contingent on the group graduating to a larger, multi-person group. The measure of this is GMAU.

Sponsorship

Anyone at GitLab can propose the creation of a single-engineer group.

The single-engineer group is prioritized just like any other new investment and needs to get approval from

- Chief Product Officer
- CTO
- CEO

Selection

If the SEG is in the Development Department, the VP of Development is responsible for deciding the reporting structure for the engineer while they are a single-engineer group.

Criteria for the engineer:

- The engineer must be a senior engineer (or above)
- They must be passionate about the subject matter
- They must be excited about the ability to work independently, or have prior success in a similar model
- Being a prior company technical co-founder
- Being an early contributor to a successful open source project
- Working successfully on a prior single-engineer group

No Stable counterparts

There are no stable counterparts for a single-engineer group. The best way to think of this is a community contribution. The selection criteria is setup to select engineers that can work independently without stable counterparts. This setup is to avoid the following undesirable side effects.

- Inadvertently asking other team members to contribute when they have their full-time responsibilities in their home group can lead to burnout and more investment than single-engineer groups are modeled to invest.
- Engineer losing the freedom because other stable counterparts may insist on more rigor around problem validation and/or solution validation.

Example for creating a Single-engineer group

The MR describes an example of how to create a single-engineer group.

Working Groups

A working group is a specific type of group that assembles for a period of time to accomplish a business goal. Working groups have defined responsibilities and meet regularly. Ideally a working group can disband when the goal is complete to avoid accrued bureaucracy.

Middle Management

Middle managers are team members who do not report to the CEO and have managers of people reporting to them.

It is not defined by someone's title or their place in the org chart, but by these two criteria.

Roles

People can be a specialist in one thing and be an expert in multiple things. These are listed on the team page.

Specialist

Specialists carry responsibility for a certain topic.

They keep track of issues in this topic and/or spend the majority of their time there.

Sometimes there is a lead in this topic that they report to.

You can be a specialist in only one topic.

The specialist description is a paragraph in the job description for a certain title.

A specialist is listed after a title, for example: Developer, database specialist (do not shorten it to Developer, database).

Many specialties represent stable counterparts. For instance, a "Software Engineer in Test, Create" specializes in the "Create" stage group and is dedicated to that group.

If you can have multiple ones and/or if you don't spend the majority of your time there it is probably an expertise.

Since a specialist has the same job description as others with the title they have the same career path and compensation.

Expert

Expert means you have above average experience with a certain topic.

Commonly, you're expert in multiple topics after working at GitLab for some time.

This helps people in the company to quickly find someone who knows more.

Please add these labels to yourself and assign the merge request to your manager.

An expertise is not listed in a role description, unlike a specialist.

For Production Engineers, a listing as "Expert" can also mean that the individual is actively embedded with another team.

Following the period of being embedded, they are experts in the regular sense of the word described above.

Developers focused on Reliability and Production Readiness are named Reliability Expert.

Mentor

Whereas an expert might assist you with an individual issue or problem, mentorship is about helping someone grow their career, functional skills, and/or soft skills. It's an investment in someone else's growth.

Some people think of expertise as hard skills (Ruby, International Employment Law, etc) rather than soft skills (managing through conflict, navigating career development in a sales organization, etc).

If you would like to be a mentor in a certain area, please add the information to the team page. It is important to note whether you would like to be a mentor internally and/or externally at GitLab. Examples of how to specify in the expertise section of the team page: Mentor - Marketing, Internal to GitLab or Mentor - Development (Ruby), External and Internal to GitLab.

Other Considerations

The word "Manager" in a title doesn't imply people management or structure

Some of individual contributors (without any direct reports) have manager in their title without a comma after it. These titles are not considered a people manager in our company structure nor salary calculator, examples are product manager, accounting manager, account manager, channel sales manager, technical account manager, field marketing managers, online marketing manager, and product marketing manager. People with manager and then a comma are a people manager in our company structure.

"Team", "team member", and "community" terminology

The term "team" is reserved for the smallest group. A team is defined as a manager and their reports. "Team" does not refer to a group or department.

We refer to all the people working for the company as "team members". This is a bit confusing, given that "team" refers to a small group, but we believe "team member" is preferable over all the alternatives we considered:

1. We don't use the term "employees" because we have many contractors working for GitLab Inc.
1. "Staff" appears on our user profile if we work for GitLab Inc., but this is confusing because it's also an engineering level.
1. "Gitlabbers" is no longer used, because it isn't inclusive of the wider community.
1. We considered calling team members "Tanuki" (referring to our logo), but it is confusing to refer to humans with an animal species.

GitLab is a project bigger than GitLab the company. It is really important that we see the community around GitLab as something that includes the people at the company. Therefore, when you use the term "community," you should be referring to all GitLab users and contributors - including team members.

To refer to users and contributors who are not team members, use "wider community."

For more, please see the GitLab writing guidelines.

SECTION: company/team.md

More than 3,000 people have contributed to GitLab. The GitLab Inc. team consists of the following {{< team-size >}} team members and their 363+ pets. We believe we're the world's largest all-remote organization and we currently have team members in more than 65 countries and regions. This page lists who people report to, and on a separate page we detail the organizational structure. You can get a sense of the team culture and our inclusion by visiting our culture page and our Identity Data page.

{{.h4}}

{{< team >}}

SECTION: company/vision.md

Vision

Our vision captures what we want to deliver to customers in the next 10 years.

Our current vision is available to GitLab team members in the internal handbook.

The remainder of this page includes details about GitLab's approach to long-term planning.

Cadence

Our Mission is on a 30-year cadence. It is the inspiration for this company vision, which is on a 10-year cadence.

The vision here is reflected in our company strategy, which is on a 3-year cadence. You can find additional product vision details on our direction page.

Monitoring an evolving market

While we define our mission, vision, and strategy, we also acknowledge that we need to adapt with a changing market to meet customer needs. Netflix is a great example of this. Everyone knew that video on demand was the future. Netflix, however, started shipping DVDs over mail. They knew that it would get them a database of content that people would want to watch on demand. In other words, timing is everything.

Additionally, we must ensure that our Platform is open. If a new, better version control technology enters the market, we will need to integrate it into our platform, as it is one component in an integrated DevSecOps product.

Entering new markets

GitLab has taken existing, fragmented software tooling markets, and by offering a consolidated offering instead, have created a new blue ocean. We would like to find more markets where we can repeat this same model.

The desirable characteristics of such markets fall into two stages: category consolidation and creation.

Category Consolidation

1. A set of customer needs that are currently served by multiple, independent software tools.
1. Those tools may already integrate with each other or have the possibility of integration.
1. Those tools operate in categories that are typically considered discrete (for example, with GitLab, SCM was one market and CI was another).
1. There is no current 'winner' at consolidating this market, even if there are some attempts to combine some of the tool categories within said market.
1. Users of the product are able to contribute to it - for example, with GitLab, many users are software developers who can directly contribute code back to the product itself.
1. When initially combined, the categories form a consistent and desirable user flow, which solves an overriding customer requirement.
1. We can offer a consolidated toolchain that is more cost effective for customers than purchasing, running, and integrating each tool separately.
1. We can do so profitably for the company.

Category Creation

1. By combining categories, a new overriding category, or market, gets created: the consolidated toolchain.
1. The sum of the categories combined should have desirable CAGR, such that entering the market means entering on a growth curve.
1. Not all of the individual categories need to be on the upward curve of the technology adoption lifecycle (TAL). It is, however, necessary that there are enough future categories with high CAGR that are early in their adoption lifecycle.
1. The ideal overlap of the TALs is that the peaks of the curves are as close to each other as possible, so that when one TAL moves to Late Majority, the next TAL is still curving upwards. This allows it to provide an organic growth opportunity for the company in the markets it is entering.

Our goal is to develop this model to be more quantifiable and formulaic, so that we can quickly and easily assess new opportunities.

Mitigating Concerns

We acknowledge the concerns to achieving our goals. We document them in our Mitigating Concerns page.

```
{{% include "includes/take-gitlab-for-a-spin.md" %}}
```

```
## SECTION: company/culture/_index.md
```

Introduction

Please see our company page for more general information about GitLab.

```
<!-- blank line -->
<figure style="padding:56.25% 0 0 0;position:relative;">
  <iframe src="https://player.vimeo.com/video/947155019" frameborder="0" allow="autoplay;
fullscreen; picture-in-picture"
style="position:absolute;top:0;left:0;width:100%;height:100%;"></iframe>
</figure>
<!-- blank line -->
```

Life at GitLab

At GitLab, our mission is simple: to enable everyone to contribute to and co-create the software that powers our world. Our platform doesn't just bring people together—it unites teams and organizations, breaking down barriers and redefining what's possible in software development.

Here, you're not just contributing to a tool, you're shaping the future of secure, scalable software delivery. Our high-performance culture values collaboration, ownership, and impact, accelerating careers while driving global innovation.

This is more than a job, it's your opportunity to build something that matters.

Everyone can contribute

Our size and our mission (enable everyone to contribute to and co-create the software that powers our world.) mean that our team members can—and are expected to—make an impact across the company.

Because we all use our product internally, you don't have to be a developer to learn to collaborate in the GitLab tool.

From your very first week, no matter your role, you'll gain the technical skills needed to access, question, and contribute to projects far beyond your job description.

This unique approach works because we're a team of helpful, passionate people who want to see each other, the company, and the broader GitLab community succeed.

We learn from each other, challenge each other, and thank each other.

Come prepared to do meaningful work that will help shape the future of the company.

While the opportunities to contribute are boundless in a growing organization like GitLab, they may not be clearly defined.

You'll need to think creatively, speak up to see how you can help, and be willing to try something new.

Freedom to iterate

At GitLab, our value of iteration has a unique impact on the way we operate and get things done.

Working this way means our team members are expected to quickly deliver the minimal valuable change in their work instead of waiting to produce a polished, completed product.

While this can be a challenging practice to adopt at first, it's liberating to be able to make mistakes, get feedback quickly, and course correct to reach a better outcome, faster.

As our company and the industry continue to grow, you'll have the freedom to change and constantly evolve everything from your schedule and your workspace to your job description and your skills.

Culture at GitLab

Culture at GitLab is composed of three things.

1. GitLab Values

1. Camaraderie (mutual trust and friendship). This is created and maintained by informal communication. We also build trust by effectively collaborating with others.

1. Work style (how we work). This is defined and articulated in our communication guide and handbook usage guide.

Other pages related to culture

1. Internal Feedback

1. Diversity, Inclusion and Belonging

1. Talent Brand

SECTION: company/culture/cross-culture-collaboration-guide.md

Goal

As an organization with 67+ countries and regions represented, it is ever-important for all of us to increase our cultural awareness, sensitivity, and understanding. This guide will help GitLab team members in effective and inclusive communication and cross-cultural collaboration globally; irrespective of culture or background.

Individual Culture vs. Company Culture

An important principle in any global organization is the distinction between our cultural background vs. the GitLab company culture. A critical aspect of our recruitment process is ensuring the people we hire are aligned with our company values.

Our diverse backgrounds help us live out our Diversity, Inclusion & Belonging value, while alignment with our company values allow us to collaborate as a team. While maintaining respect for each team member's individual culture, GitLab culture and values should be what unites us. For many team members, some company values and the digital and asynchronous way in which we work at GitLab may take some adjusting to during onboarding. We should support and encourage each other to assume positive intent and live out our values daily!

Research

There has been much research done on how to understand the cultural dimensions of a country/region and the impact different dimensions have on communication and collaboration. In our table below, we have provided an overview from two researchers who have spent years dedicated to defining different cultural dimensions. One of the most well-known pioneer studies conducted on cross-cultural groups and organizations was by Geert Hofstede in his book *Cultures and Organizations: Software of the Mind*. A more modern study that was published in 2014 also exploring cross-cultural collaboration in the workplace was conducted by Erin Meyer in her book *The Culture Map*. GitLab based our cultural dimensions on the research from these two experts, and aligned our dimensions to our values tailored to our company to help foster understanding and communication.

If you are interested in learning more about the research that inspired GitLab, you can review Hofstede's cultural dimensions in more detail in addition to his cultural dimension comparison tool to view how specific countries compare, or considering reading his book *Cultures and Organizations: Software of the Mind*. If you are interested in learning more about Erin Meyer's cultural dimensions, you can view a summary in this article, or read her book *The Culture Map*.

It is important to note that while the dimensions listed below can help us understand general orientations and trends in several countries based on research, these dimensions are not absolute. Each person is unique and thus the dimensions are relative.

Cultural Dimensions

Cultural Relativity

Keep in mind that for all of the dimensions defined below, we need to be mindful of cultural relativity. What matters is not so much any given country's tendency, but rather, their positioning in relationship to you. For example, the United Kingdom is one of the lowest context cultures. However, in comparison to the United States, which is the lowest context culture, the British could be interpreted as high context in their communication.

Cultural Dimensions Table

Note that for the Values Alignment section in the table below, Diversity, Inclusion & Belonging is not explicitly listed. This is because becoming culturally aware and empathetic in our interactions in and of itself supports our Diversity, Inclusion & Belonging value. Thus, reviewing the table below and implementing awareness in your day-to-day interactions at GitLab demonstrates Diversity, Inclusion & Belonging.

|GitLab | Meaning | Values Alignment |

| ----- | ----- | ----- |

| Leadership Expectation | This dimension refers to the difference between egalitarian and hierarchical leadership styles. Countries that follow egalitarian models prefer "flat" organizational structures, making the distance between a manager and a team member low. Hierarchical cultures have a significant distance between manager and team member; status is very important and communication follows hierarchical flow. | Collaboration |

| Navigating Disagreement | The level of comfort and acceptance in regard to or Engaging or Disengaging in disagreement/debate varies drastically from culture to culture. Engaging

cultures believe that healthy debate is positive for teams and appropriate, while Disengaging cultures believe that disagreement has a negative impact on team dynamic, disrupts harmony, and hurts the relationships overall. GitLab advocates for constructive passionate debate and encourages us to have "short toes". | Collaboration, Efficiency |

| Feedback Approach | Direct vs. indirect feedback, specifically in regard to improvement areas, can make a big difference in the interpretation of messages we try to relay, or feedback we give. While we advocate for transparency and are public by default, it is important to be mindful that some cultures can be less receptive to direct and public feedback than others. In direct feedback cultures, feedback on improvement areas is provided in a straightforward manner bluntly and is not softened by other positive feedback. It is acceptable for feedback to be provided in public settings. In indirect feedback cultures, feedback on improvement areas tends to be given subtly and softly and is generally "sandwiched" between positive feedback. Feedback is only given in private settings. | Transparency |

| Communication Style | While GitLab advocates low-context communication (precision, simplicity, and clarity), it is important to be aware that many cultures are more accustomed to high-context communication (layered communication that is more "read between the lines"; messages are often implied versus directly expressed). We should be tolerant of all communication styles and ask for clarification when needed. | Iteration, Results |

| Collaboration Focus | Task-oriented cultures tend to focus on the task at hand with speed and efficiency to produce results as soon as possible. Relationship oriented cultures tend to focus not only on the task at hand but also in cultivating and developing relationships to increase collaboration and efficiency as a team. This may be harder to recognize in an asynchronous environment where there is less face to face interaction to build these relationships. |

Iteration, Collaboration |

| Concept of Time | Different cultures abide by different concepts of time: linear time and flexible time. Characteristics of linear time include being punctual, following a set agenda, and starting and ending as scheduled. Flexible time cultures tend to put relationships before timeliness, are more flexible in their use of agendas, and will wait a few minutes for everyone to arrive. GitLab's sense of time tends to be more linear than flexible, we define being respectful of others time as including an agenda, documenting the outcome, and starting promptly (we don't wait if people arrive after the schedule start time). | Results, Efficiency |

|

Practices For Cultural Awareness

While the table above outlines GitLab's cultural dimensions, meaning, and values alignment, this section aims to provide a more practical "how-to" as a platform for applying the awareness the dimensions provide to every day interactions at GitLab. While the practices below can and should be implemented in all interactions, they become particularly important in cross-cultural communication.

Stay Values-Focused

It is the responsibility of every GitLab team member to live out our values. As highlighted above, GitLab culture and values are what unite us; our values provide common ground irrespective of our backgrounds. We should remind each other and ourselves to stay values-centered and let our values guide our interactions!

Assume Positive Intent

An operating principle of collaboration, assuming positive intent is also critical for effective cross-cultural communication. Not everyone has worked in global environments before, and as humans, we all make mistakes. If someone says something that rubs you the wrong way, assume positive intent, explain how you feel in a 1:1 setting, and ask for clarification to ensure you have the full picture. We should give each other the benefit of the doubt in interactions.

Ask Questions

A lot of people love to share about their cultures! When in doubt, ask. Asking questions results in learning, which helps make us all more culturally aware.

Be Humble

Recognize that your beliefs are not absolute. There are 195 countries in the world, 4,200 different religions, and 7.5 billion people; each one of us has a unique set of customs and culture, even if our cultural backgrounds are similar. Awareness that different beliefs exist is a critical step in practicing effective cross-cultural communication.

Be Empathetic

Be empathetic towards others, their reactions, and their culture. There is often more than meets the eye, and it is important to remember that something "normal" to one person may not be normal to the other.

Set Expectations

Clearly set expectations around communication in order to establish and align on what works best for your team dynamic and the cultures within your team. Creating this awareness and alignment can help avoid misunderstandings and ensure cultural and personal backgrounds are respected.

Stories from Team Members

Discussing Cross-Cultural Communication

Listen to the panel discussion on cross-cultural communication below hosted on 2022-03-23 with GitLab team members.

{{< youtube "XS4HhVLF7o" >}}

Key Session Takeaways

1. In remote spaces, intentionality is important to build trust on teams. Ideas on how to do this include adding a social question to team meetings or hosting a social event to share about people's localities.
1. Assuming positive intent when communicating async can help us recognize and have compassion for other's experiences.
1. Everyone can be an ally and has a responsibility to check and fix their own biases when collaborating with others

SECTION: company/culture/ecoc.md

GitLab is dedicated to providing a harassment-free and inclusive event experience for everyone regardless of gender identity or expression, sexual orientation, disability, neurodiversity, physical appearance, body size, ethnicity, nationality, race, age, religion (or lack thereof), or any other categories protected by local, state, provincial, or federal law. We do not tolerate harassment against any individual at our events in any form. GitLab takes violations of this policy seriously and event organizers will enforce our policies throughout the event.

This policy extends to:

- Talks
- Forums
- Workshops
- Social media (or other online media), including personal accounts
- Parties
- Hallway conversations
- Booths and other similar activities
- Virtual Environments
- Electronic communication - via Slack, email, or event apps

This